BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

VİZE PROJE RAPORU

Proje 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı
Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması

AD SOYAD: Hüseyin TINAZTEPE
NUMARA: 21290360

GITHUB:
https://github.com/hsyntinaztepe/DB-Backup-Recovery-Automation-Prj2-Prj7

VERİTABANI: AdventureWorks2022
TARİH: Nisan 2026

İÇİNDEKİLER

1.1 Giriş ve Proje Amacı....3
1.2 Veritabanı Hazırlığı – Recovery Model....3
1.2.1 Recovery Model Ayarı.... 3
1.3 Yedekleme Stratejisi (Full / Differential / Log)....3
1.3.1 Tam Yedekleme.... 4
1.3.2 Fark Yedekleme....4
1.3.3 Transaction Log Yedekleme.... 4
1.4 Felaket Senaryosu – Veritabanının Silinmesi.... 4
1.5 Felaketten Kurtarma – Restore İşlemi.... 4
1.5.1 Restore Zinciri.... 4
1.5.2 Restore Sonrası Kontrol.... 5
1.6 Point-in-Time Restore.... 5
1.6.1 Senaryo Hazırlığı....5
1.6.2 Point-in-Time Restore Uygulaması....5
1.7 Yedek Doğrulama.... 6
1.8 Yedekleme Geçmişi Raporlaması.... 6
1.9 SQL Sorgu Dosyaları Özeti....6
1.10 Sonuç ve Değerlendirme.... 7
2.1 Giriş ve Proje Amacı....8
2.2 BackupLog Tablosu Oluşturma.... 8
2.3 Manuel Tam Yedekleme.... 8
2.4 sp_AutoBackup – Otomatik Yedekleme Prosedürü....9
2.4.1 Prosedür Testi....9
2.5 SQL Server Agent Job Kurulumu.... 10
2.5.1 Job'un Manuel Çalıştırılması.... 10
2.5.2 Job Geçmişinin Kontrolü....10
2.6 Hata Senaryosu ve Log Kaydı....10
2.7 Raporlama Sorguları.... 11
2.7.1 Günlük Özet Raporu.... 11
2.7.2 Son 10 Yedek....11
2.7.3 msdb Yedekleme Geçmişi.... 11
2.8 Yedek Doğrulama....11
2.9 SQL Sorgu Dosyaları Özeti....12
2.10 Sonuç ve Değerlendirme.... 12

Proje 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı

1.1 Giriş ve Proje Amacı

Bu projede Microsoft tarafından yayımlanan AdventureWorks2022 örnek veritabanı kullanılmıştır. AdventureWorks, kurgusal bir bisiklet üretim şirketine ait satış, üretim, insan kaynakları ve müşteri verilerini barındıran kapsamlı bir veritabanıdır. İçeriğinde Person, Sales, Production, HumanResources ve Purchasing gibi şemalar altında 70'ten fazla tablo, çok sayıda view ve stored procedure bulunmaktadır. Yaklaşık 200 MB boyutuyla gerçek bir kurumsal veritabanını simüle etmekte olup yedekleme ve kurtarma senaryolarını test etmek için ideal bir yapı sunmaktadır.

Projenin temel amacı, kurumsal bir veritabanı için kapsamlı bir yedekleme stratejisi oluşturmak ve felaketten kurtarma (Disaster Recovery) senaryolarını uygulamalı olarak gerçekleştirmektir:

- Tam (Full), Fark (Differential) ve Log yedeklemelerinin planlanması ve uygulanması
- Kaza ile silinen veya bozulan veritabanının yedeklerden geri yüklenmesi
- Point-in-Time Restore ile belirli bir zaman noktasına geri dönme
- Yedek dosyalarının doğrulanması ve raporlanması

1.2 Veritabanı Hazırlığı – Recovery Model

1.2.1 Recovery Model Ayarı

Point-in-Time Restore ve Log yedeklemesi yapabilmek için veritabanının Recovery Model'inin FULL olması zorunludur. Aşağıdaki komutlarla bu ayar yapılmış ve doğrulanmıştır:

```
USE master;
ALTER DATABASE AdventureWorks2022 SET RECOVERY FULL;
ALTER DATABASE AdventureWorks2022 SET MULTI_USER;
SELECT name, recovery_model_desc, user_access_desc
FROM sys.databases WHERE name = 'AdventureWorks2022';
```

Sorgu sonucunda recovery_model_desc sütununda FULL değeri görülmektedir. FULL recovery model, her işlemin transaction log'a yazılmasını sağlar; bu sayede Point-in-Time Restore mümkün olur.

1.3 Yedekleme Stratejisi (Full / Differential / Log)

Proje kapsamında üç farklı yedekleme türü uygulanmıştır. Bu strateji kurumsal ortamlarda yaygın kullanılan Full + Differential + Log yedekleme zincirini oluşturmaktadır.

Yedek Türü	Dosya	Amaç	Query
Tam Yedek (Full)	AW_Full.bak	Tüm veritabanını yedekler	Query2_FullBackup.sql
Fark Yedek (Diff)	AW_Diff.bak	Son Full'dan değişenleri kaydeder	Query3_DiffBackup.sql
Log Yedek	AW_Log1/2/3.bak	Transaction log'u yedekler	Query4_LogBackup.sql

1.3.1 Tam Yedekleme

Tam yedekleme tüm veriyi ve yapıyı tek dosyaya kaydeder; kurtarma zincirinin başlangıç noktasıdır:

```
BACKUP DATABASE AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Full.bak'
WITH FORMAT, INIT, NAME = 'AW Full Backup', STATS = 10;
```

Yedek boyutu yaklaşık 200 MB olarak gerçekleşmiştir.

1.3.2 Fark Yedekleme

Fark yedeklemesi son tam yedekten bu yana değişen sayfaları kaydeder. Önce Person.Address tablosunda güncelleme yapılmış, ardından diferansiyel yedek alınmıştır:

```
UPDATE Person.Address SET ModifiedDate = GETDATE()
WHERE AddressID BETWEEN 1 AND 100;

BACKUP DATABASE AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Diff.bak'
WITH DIFFERENTIAL, NAME = 'AW Differential Backup', STATS = 10;
```

Diferansiyel yedek boyutu yaklaşık 2 MB olup yalnızca değişen verilerin yedeklendiğini kanıtlamaktadır.

1.3.3 Transaction Log Yedekleme

Log yedeklemesi transaction günlüğünü temizleyerek Point-in-Time Restore imkânı tanır:

```
BACKUP LOG AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Log1.bak'
WITH NAME = 'AW Log Backup 1', STATS = 10;
```

1.4 Felaket Senaryosu – Veritabanının Silinmesi

Query5_Felaket.sql dosyasında gerçek bir felaket senaryosu simüle edilmiştir. Veritabanı kasıtlı olarak silinerek kurtarma sürecinin etkinliği test edilmiştir:

```
SELECT GETDATE() AS FelaketOncesiZaman;
ALTER DATABASE AdventureWorks2022 SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
```

```
DROP DATABASE AdventureWorks2022;
SELECT name FROM sys.databases WHERE name = 'AdventureWorks2022';
```

DROP DATABASE komutu sonrasında veritabanı tamamen silinmiştir. sys.databases sorgusunda AdventureWorks2022 artık görünmemekte; sorgu boş dönmektedir.

1.5 Felaketten Kurtarma – Restore İşlemi

1.5.1 Restore Zinciri

Silinen veritabanını geri yüklemek için Full → Differential → Log sırasıyla backup uygulanmıştır. NORECOVERY zincirin devam edeceğini; son adımdaki RECOVERY ise veritabanını açacağını belirtir:

```
RESTORE DATABASE AdventureWorks2022
FROM DISK = 'C:\Backup\AdventureWorks\AW_Full.bak'
WITH NORECOVERY, REPLACE, STATS = 10;

RESTORE DATABASE AdventureWorks2022
FROM DISK = 'C:\Backup\AdventureWorks\AW_Diff.bak'
WITH NORECOVERY, STATS = 10;

RESTORE LOG AdventureWorks2022
FROM DISK = 'C:\Backup\AdventureWorks\AW_Log1.bak'
WITH RECOVERY, STATS = 10;
```

1.5.2 Restore Sonrası Kontrol

```
SELECT name, state_desc, recovery_model_desc FROM sys.databases
WHERE name = 'AdventureWorks2022';
SELECT COUNT(*) AS ToplamTablo FROM INFORMATION_SCHEMA.TABLES;
SELECT TOP 5 * FROM Person.Address;
```

Sorgu sonuçlarında state_desc = ONLINE ve tüm tabloların yerinde olduğu görülmüştür. Kurtarma işlemi başarıyla tamamlanmıştır.

1.6 Point-in-Time Restore

1.6.1 Senaryo Hazırlığı

- AW_Log2.bak alındı — bu noktaya geri dönülecek
- Person.Address tablosundaki ilk 10 kayıt kasıtlı olarak bozuldu
- Bozulma sonrası AW_Log3.bak alındı

```
BACKUP LOG AdventureWorks2022
TO DISK = 'C:\Backup\AdventureWorks\AW_Log2.bak';

UPDATE Person.Address
SET AddressLine1 = 'BOZUK ADRES ' + CAST(AddressID AS NVARCHAR)
WHERE AddressID BETWEEN 1 AND 10;

BACKUP LOG AdventureWorks2022
```

```
TO DISK = 'C:\Backup\AdventureWorks\AW_Log3.bak';
```

1.6.2 Point-in-Time Restore Uygulaması

STOPAT parametresiyle bozulma öncesi zaman noktasına geri dönülmüştür:

```
RESTORE DATABASE AdventureWorks2022
FROM DISK = 'AW_Full.bak' WITH NORECOVERY, REPLACE;
RESTORE DATABASE AdventureWorks2022
FROM DISK = 'AW_Diff.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log1.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log2.bak' WITH NORECOVERY;
RESTORE LOG AdventureWorks2022
FROM DISK = 'AW_Log3.bak'
WITH RECOVERY, STOPAT = '2026-04-13 22:10:00';
```

Restore sonrası Person.Address tablosu kontrol edilmiş; AddressLine1 değerlerinin orijinal haline döndüğü doğrulanmıştır.

1.7 Yedek Doğrulama

RESTORE VERIFYONLY

```
RESTORE VERIFYONLY FROM DISK = 'C:\Backup\AdventureWorks\AW_Full.bak';
```

RESTORE HEADERONLY

Yedek dosyasının meta verilerini listeler: yedek türü, alınma zamanı, SQL Server versiyonu.

RESTORE FILELISTONLY

Yedek içindeki .mdf ve .ldf dosyalarını listeler; MOVE parametresi için kullanılır.

Üç komut da başarıyla tamamlanmış; yedek dosyasının geçerli ve restore edilebilir nitelikte olduğu onaylanmıştır.

1.8 Yedekleme Geçmişi Raporlaması

```
SELECT database_name AS VeritabaniAdi,
       backup_start_date AS BaslangicZamani,
       backup_finish_date AS BitisZamani,
       DATEDIFF(SECOND, backup_start_date, backup_finish_date) AS SureSaniye,
       CASE type WHEN 'D' THEN 'Tam Yedek'
                 WHEN 'I' THEN 'Fark Yedek'
                 WHEN 'L' THEN 'Log Yedek' END AS YedekTuru,
       CAST(backup_size/1024/1024 AS INT) AS BoyutMB
FROM backupset WHERE database_name = 'AdventureWorks2022'
ORDER BY backup_start_date DESC;
```

1.9 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
Query1_RecoveryModel.sql	Recovery Model FULL yapma ve kontrol	Hazırlık
Query2_FullBackup.sql	Tam yedekleme alma	Full Backup
Query3_DiffBackup.sql	Veri değişikliği + fark yedekleme	Diff Backup
Query4_LogBackup.sql	Transaction log yedekleme	Log Backup
Query5_Felaket.sql	Veritabanını silme (felaket simülasyonu)	Felaket
Query6_Restore.sql	Full + Diff + Log ile geri yükleme	Kurtarma
Query7_Kontrol.sql	Restore sonrası sağlık kontrolü	Doğrulama
Query8_PointInTime.sql	Veri bozma + log yedekleme	PIT Hazırlık
Query8b_PointInTime_Restore.sql	STOPAT ile belirli zamana geri dönme	PIT Restore
Query9_Dogrulama.sql	VERIFYONLY, HEADERONLY, FILELISTONLY	Doğrulama
Query10_Rapor.sql	Yedekleme geçmişi raporu (msdb)	Raporlama

1.10 Sonuç ve Değerlendirme

- FULL recovery model etkinleştirilerek tüm kurtarma seçeneklerine zemin hazırlanmıştır.
- Full, Differential ve Log yedeklemeleri alınarak kurumsal bir yedekleme zinciri oluşturulmuştur.
- DROP DATABASE ile felaket senaryosu simüle edilmiş ve yedeklerden tam kurtarma gerçekleştirilmiştir.
- Point-in-Time Restore ile kasıtlı bozulan veriler belirli bir zaman noktasına döndürülmüştür.
- VERIFYONLY, HEADERONLY ve FILELISTONLY ile yedek bütünlüğü doğrulanmıştır.
- msdb.dbo.backupset sorgulanarak tüm yedekleme geçmişi raporlanmıştır.

Proje 7 – Veritabanı Yedekleme ve Otomasyon Çalışması

2.1 Giriş ve Proje Amacı

Bu projede de Proje 2 ile aynı AdventureWorks2022 veritabanı kullanılmıştır. AdventureWorks, Microsoft'un yayımladığı kurgusal bir bisiklet üretim şirketine ait örnek veritabanı olup satış, üretim, müşteri ve insan kaynakları verilerini barındırmaktadır. 70'ten fazla tablo ve yaklaşık 200 MB boyutuyla gerçek kurumsal ortamları simüle eden bu yapı, otomasyon ve raporlama senaryolarını test etmek için tercih edilmiştir. Veritabanı Microsoft SQL Server 2022 Developer Edition üzerinde çalıştırılmıştır.

- BackupLog tablosuyla yedekleme geçmişini kayıt altına almak
- sp_AutoBackup prosedürüyle Full ve Differential yedeklemeleri otomatikleştirmek
- SQL Server Agent Job ile yedeklemeyi her gece 02:00'de otomatik çalıştırmak
- Hata senaryosu oluşturup hata kayıtlarını log tablosunda gözlemlemek
- Raporlama sorguları ile yedekleme istatistiklerini analiz etmek

Not: Proje sürecinde önce SQL Server Express Edition kullanılmış; ancak SQL Server Agent'ın Express'te desteklenmediği anlaşılınca Developer Edition'a geçilmiştir.

2.2 BackupLog Tablosu Oluşturma

Yedekleme işlemlerinin sonuçlarını kayıt altına almak için BackupLog tablosu oluşturulmuştur:

```
USE AdventureWorks2022;
GO
CREATE TABLE BackupLog (
    LogID       INT IDENTITY PRIMARY KEY,
    BackupDate  DATETIME DEFAULT GETDATE(),
    FilePath    NVARCHAR(500),
    Status      NVARCHAR(20),
    FileSizeMB  FLOAT,
    DurationSec INT,
    ErrorMsg    NVARCHAR(1000)
);
```

Her yedekleme işlemi tamamlandığında bu tabloya bir satır eklenmektedir. Status alanı 'BAŞARILI' veya 'HATA' değerlerinden birini alır; hata durumunda ErrorMsg sütununa açıklama yazılır.

2.3 Manuel Tam Yedekleme

Otomasyondan önce C:\Backups klasörünün erişilebilir olduğunu doğrulamak amacıyla manuel bir tam yedek alınmıştır:

```
USE AdventureWorks2022;
GO
BACKUP DATABASE AdventureWorks2022
TO DISK = 'C:\Backups\AW_Full_Manuel.bak'
WITH FORMAT, INIT, NAME = 'Manuel Tam Yedek', STATS = 10;
```

'BACKUP DATABASE successfully processed' mesajı alınmış ve AW_Full_Manuel.bak dosyasının oluştuğu gözlemlenmiştir. Dosya boyutu yaklaşık 200 MB'tır.

2.4 sp_AutoBackup – Otomatik Yedekleme Prosedürü

Hem Full hem Differential yedeklemeleri tek prosedürden yönetmek için sp_AutoBackup oluşturulmuştur. @BackupType parametresine göre davranışını değiştirir ve sonucu BackupLog'a kaydeder:

```
CREATE PROCEDURE sp_AutoBackup
    @BackupType NVARCHAR(10) = 'FULL'
AS BEGIN
    DECLARE @path NVARCHAR(500), @date NVARCHAR(50)
    DECLARE @start DATETIME = GETDATE(), @size FLOAT

    SET @date = REPLACE(REPLACE(CONVERT(NVARCHAR,GETDATE(),120),':','-'),' ','_')
    SET @path = 'C:\Backups\AW_' + @BackupType + '_' + @date + '.bak'

    BEGIN TRY
        IF @BackupType = 'FULL'
            BACKUP DATABASE AdventureWorks2022 TO DISK=@path WITH FORMAT,
STATS=10;
        ELSE
            BACKUP DATABASE AdventureWorks2022 TO DISK=@path WITH DIFFERENTIAL,
STATS=10;

        INSERT INTO BackupLog(FilePath, Status, FileSizeMB, DurationSec)
        VALUES(@path, 'BAŞARILI', @size, DATEDIFF(SECOND,@start,GETDATE()));
    END TRY
    BEGIN CATCH
        INSERT INTO BackupLog(FilePath, Status, DurationSec, ErrorMsg)
        VALUES(@path, 'HATA', DATEDIFF(SECOND,@start,GETDATE()),
ERROR_MESSAGE());
    END CATCH
END;
```

2.4.1 Prosedür Testi

```
EXEC sp_AutoBackup @BackupType = 'FULL';
EXEC sp_AutoBackup @BackupType = 'DIFF';
SELECT * FROM BackupLog;
```

Test sonucunda C:\Backups klasöründe tarih damgalı iki .bak dosyası oluşmuş, BackupLog tablosuna 'BAŞARILI' durumuyla iki kayıt eklenmiştir. Full yedek ~200 MB, Differential yedek ~2 MB boyutundadır.

2.5 SQL Server Agent Job Kurulumu

sp_AutoBackup prosedürünün her gece 02:00'de otomatik çalışması için SQL Server Agent Job oluşturulmuştur:

```
USE msdb;
GO
EXEC sp_add_job @job_name = N'AW_Otomatik_Yedekleme';

EXEC sp_add_jobstep
    @job_name      = N'AW_Otomatik_Yedekleme',
    @step_name     = N'Tam Yedek Al',
    @subsystem     = N'TSQL',
    @database_name = N'AdventureWorks2022',
    @command       = N'EXEC sp_AutoBackup @BackupType = ''FULL''';

EXEC sp_add_schedule
    @schedule_name     = N'HerGunSaat2',
    @freq_type         = 4,
    @freq_interval     = 1,
    @active_start_time = 020000;

EXEC sp_attach_schedule @job_name=N'AW_Otomatik_Yedekleme',
@schedule_name=N'HerGunSaat2';
EXEC sp_add_jobserver  @job_name=N'AW_Otomatik_Yedekleme';
```

2.5.1 Job'un Manuel Çalıştırılması

```
USE msdb;
EXEC sp_start_job N'AW_Otomatik_Yedekleme';
```

2.5.2 Job Geçmişinin Kontrolü

```
SELECT j.name AS JobAdi, h.run_date AS Tarih, h.run_time AS Saat,
       CASE h.run_status WHEN 0 THEN 'HATA'
                         WHEN 1 THEN 'BAŞARILI'
                         WHEN 3 THEN 'İPTAL' END AS Durum,
       h.message AS Mesaj
FROM sysjobs j JOIN sysjobhistory h ON j.job_id = h.job_id
WHERE j.name = 'AW_Otomatik_Yedekleme'
ORDER BY h.run_date DESC, h.run_time DESC;
```

Sorgu sonucunda Job'un BAŞARILI durumuyla çalıştığı görülmüştür. 'The job succeeded. The job was invoked by User' mesajı başarılı çalışmanın kanıtıdır.

2.6 Hata Senaryosu ve Log Kaydı

Prosedürün hata durumundaki davranışını test etmek için C:\Backups klasörü C:\Backups_old olarak yeniden adlandırılmış ve sp_AutoBackup çalıştırılmıştır. SQL Server hedef klasörü bulamadığından yedekleme başarısız olmuştur.

Hata mesajı: 'BACKUP DATABASE is terminating abnormally.'

BackupLog tablosunda Status = 'HATA' ve ErrorMsg sütununda hata açıklaması gözlemlenmiştir. Klasör geri adlandırılıp prosedür tekrar çalıştırıldığında 'BAŞARILI' kaydı düşmüştür.

LogID	Durum	FileSizeMB	Açıklama
1, 3, 6–9	BAŞARILI	~200 MB / ~2 MB	Normal yedeklemeler
5	HATA	NULL	Klasör bulunamadı – terminating abnormally

2.7 Raporlama Sorguları

2.7.1 Günlük Özet Raporu

```
SELECT CONVERT(DATE, BackupDate) AS Tarih,
       COUNT(*) AS ToplamYedek,
       SUM(CASE WHEN Status='BAŞARILI' THEN 1 ELSE 0 END) AS Basarili,
       SUM(CASE WHEN Status='HATA'     THEN 1 ELSE 0 END) AS Hatali,
       AVG(FileSizeMB) AS OrtBoyutMB, AVG(DurationSec) AS OrtSureSaniye
FROM BackupLog
GROUP BY CONVERT(DATE, BackupDate) ORDER BY Tarih DESC;
```

2.7.2 Son 10 Yedek

```
SELECT TOP 10 * FROM BackupLog ORDER BY BackupDate DESC;
```

2.7.3 msdb Yedekleme Geçmişi

```
SELECT TOP 10 database_name, backup_start_date, backup_finish_date,
       backup_size/1024/1024 AS SizeMB,
       CASE type WHEN 'D' THEN 'Tam Yedek'
                 WHEN 'I' THEN 'Diferansiyel'
                 WHEN 'L' THEN 'Log Yedeği' END AS YedekTipi
FROM msdb.dbo.backupset WHERE database_name = 'AdventureWorks2022'
ORDER BY backup_finish_date DESC;
```

2.8 Yedek Doğrulama

```
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\AW_Full_Manuel.bak';
```

Komut 'The backup set on file 1 is valid.' mesajıyla tamamlanmıştır. Yedek dosyasının bütün, okunabilir ve restore edilebilir nitelikte olduğu garanti edilmiştir.

2.9 SQL Sorgu Dosyaları Özeti

Dosya Adı	İçerik	Konu
Query1_BackupLog_Tablo.sql	BackupLog tablosunu oluşturma	Tablo
Query2_Manuel_FullBackup.sql	Manuel tam yedek alma	Full Backup
Query3_AutoBackup_Procedure.sql	sp_AutoBackup prosedürü	Otomasyon
Query4_Procedure_Test.sql	Full ve Diff prosedür testi	Test
Query5_AgentJob_Olustur.sql	Agent Job ve zamanlama kurulumu	Agent Job
Query6_AgentJob_Calistir.sql	Job'u manuel tetikleme	Çalıştırma
Query7_Job_Gecmis_Kontrol.sql	Job geçmişi sorgulama	Kontrol
Query8_Raporlama.sql	Günlük özet, son 10, msdb raporu	Raporlama
Query9_Dogrulama.sql	RESTORE VERIFYONLY	Doğrulama
Query10_BackupLog_Goruntule.sql	BackupLog tablosunu görüntüleme	Görüntüleme
Query11_VerifyOnly_Dogrula.sql	Son yedek dosyasını doğrulama	Doğrulama

2.10 Sonuç ve Değerlendirme

- BackupLog tablosu ile tüm yedekleme işlemlerinin başarı/hata durumu kayıt altına alınmıştır.
- sp_AutoBackup prosedürü sayesinde Full ve Differential yedeklemeler tek komutla yönetilebilir hale gelmiştir.
- SQL Server Agent Job ile yedekleme günlük 02:00'de otomatik çalışacak şekilde zamanlanmıştır.
- Hata senaryosunda prosedürün hatayı yakalayıp log tablosuna kaydettiği doğrulanmıştır.
- Günlük özet ve msdb geçmiş sorguları ile yedekleme istatistikleri raporlanmıştır.
- RESTORE VERIFYONLY komutuyla yedek dosyalarının geçerliliği teyit edilmiştir.